

Verifying Post-Compromise Security (with Application Controls) in ProVerif

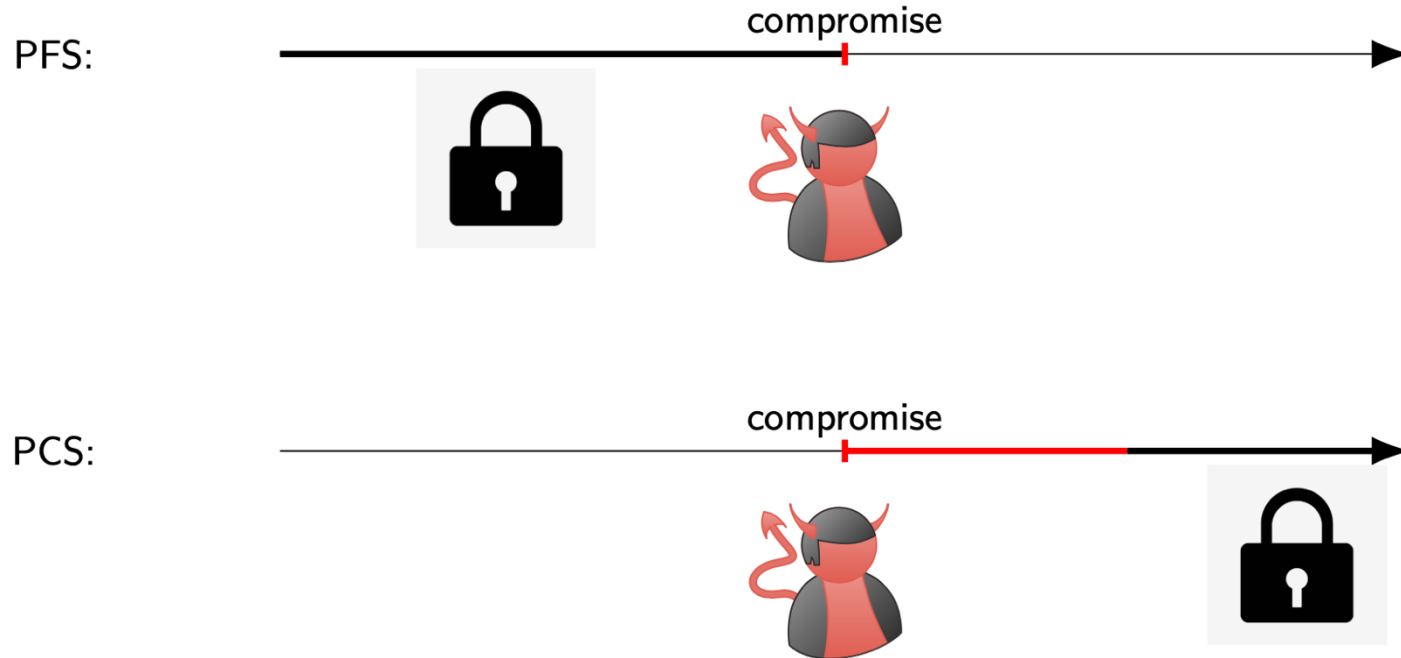
Ioana Boureanu @ “Workshop: 25 years of ProVerif”

3rd of June 2026

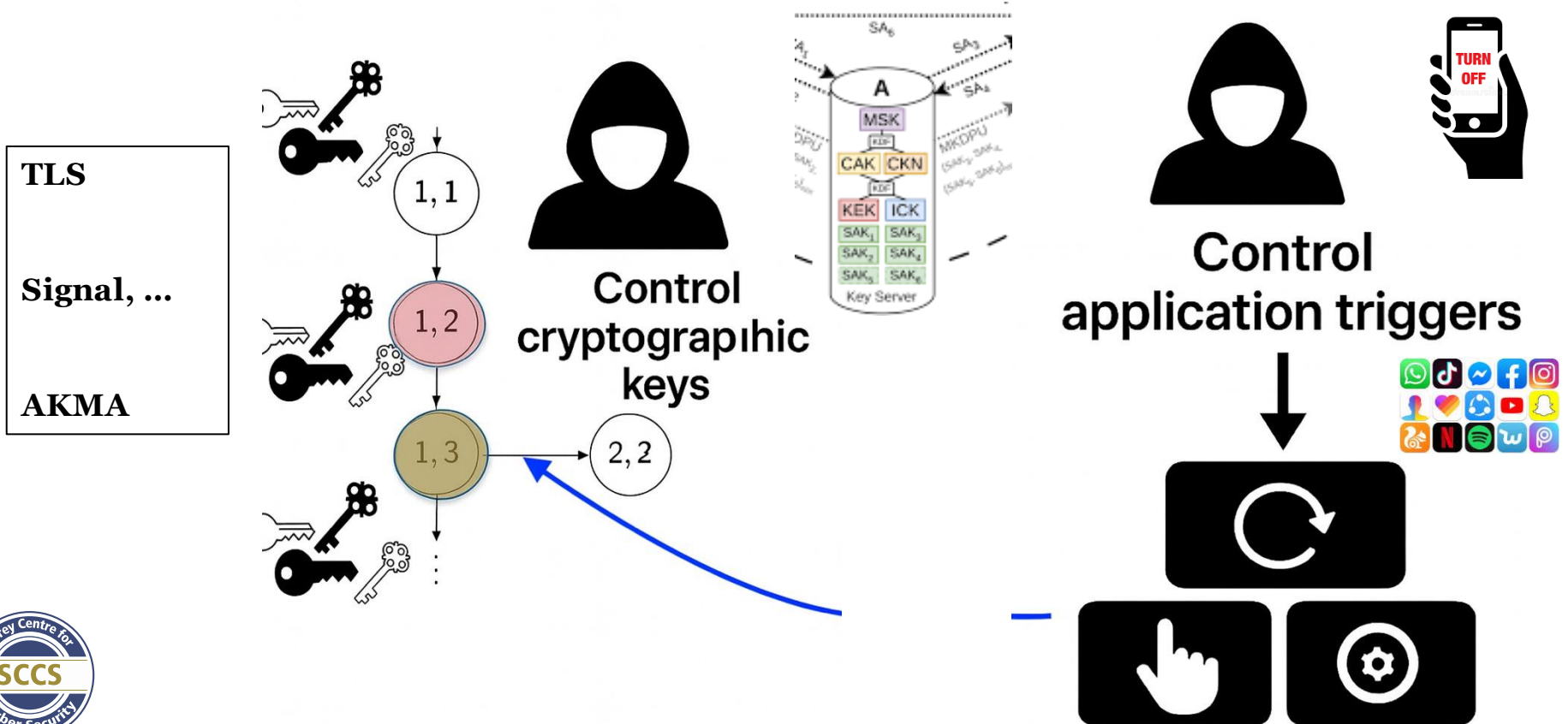
INRIA, Paris

*based on papers at USENIX2023 & at ASIACCS2025; joint work with
Olivier Blazy, Cristina Onete, Stephan Wesemeyer, Léo Robert, Rhys Miller, Pascal Lafourcade, ...*

Post-compromise Security (PCS)

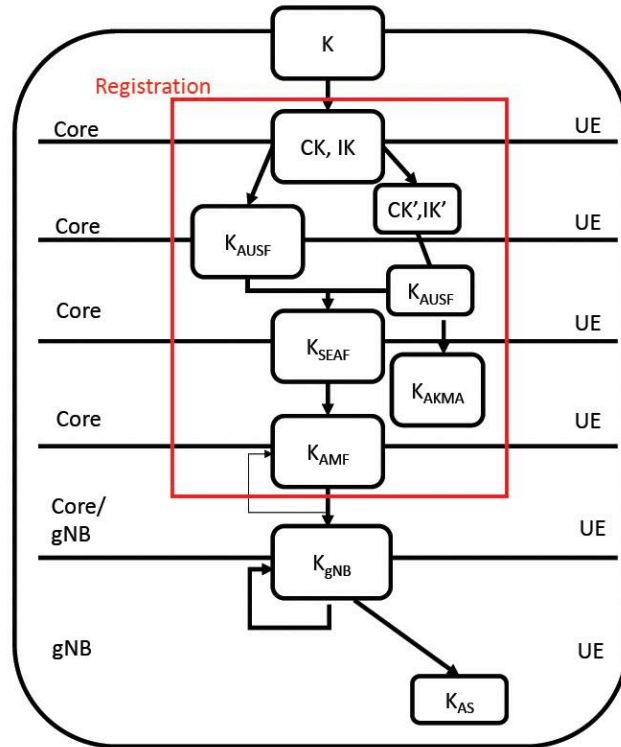


PCS with Application Controls/Triggers

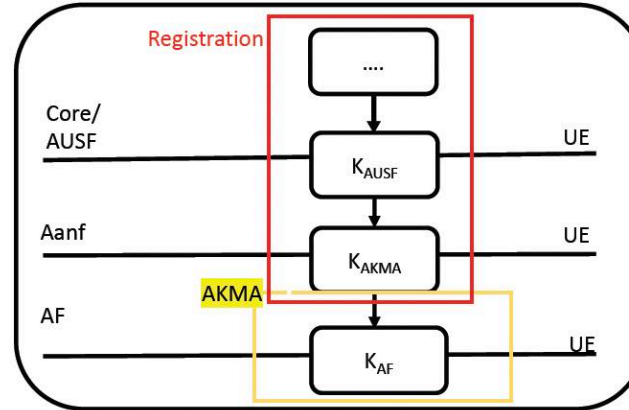


Where does PCS appear?

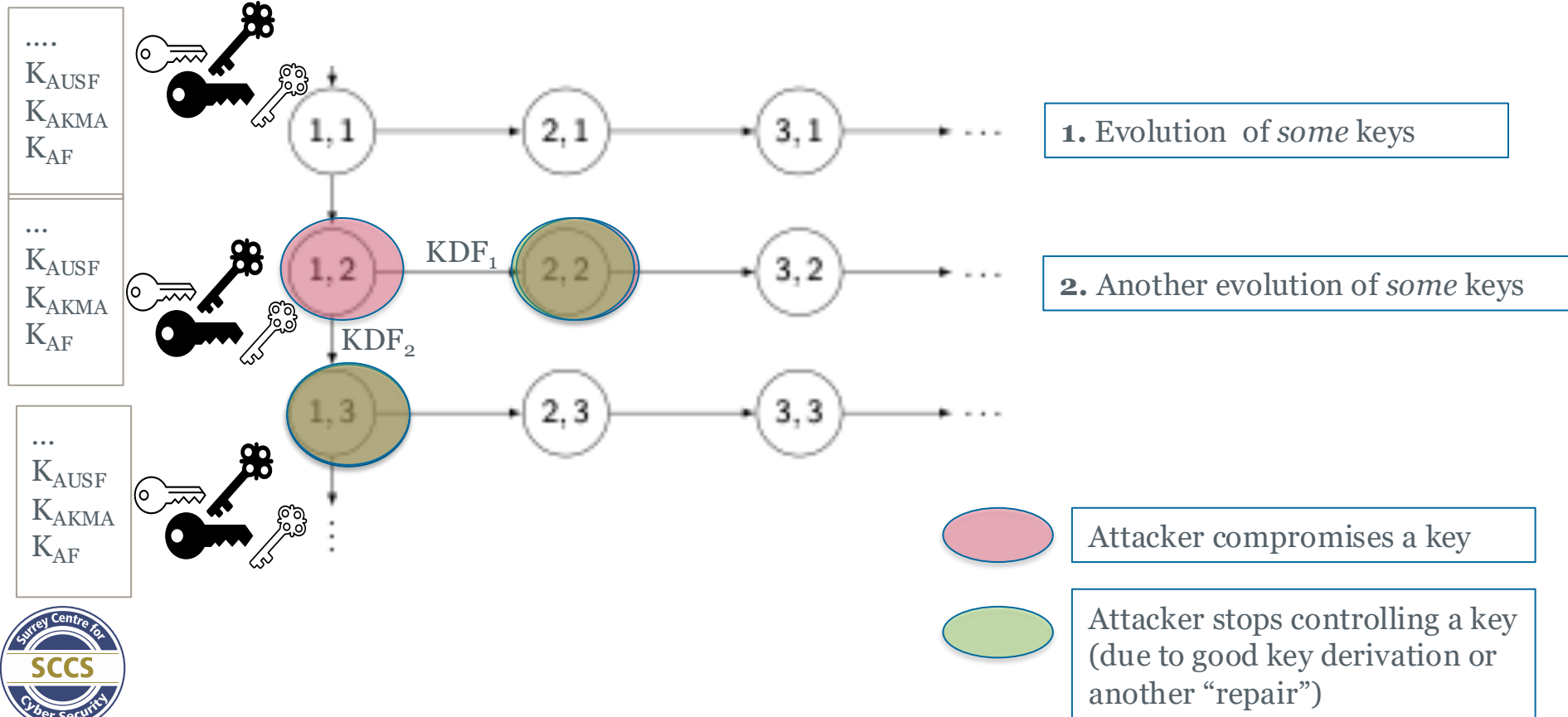
E.g., 3GPP and 5G Protocols' Key Hierarchy



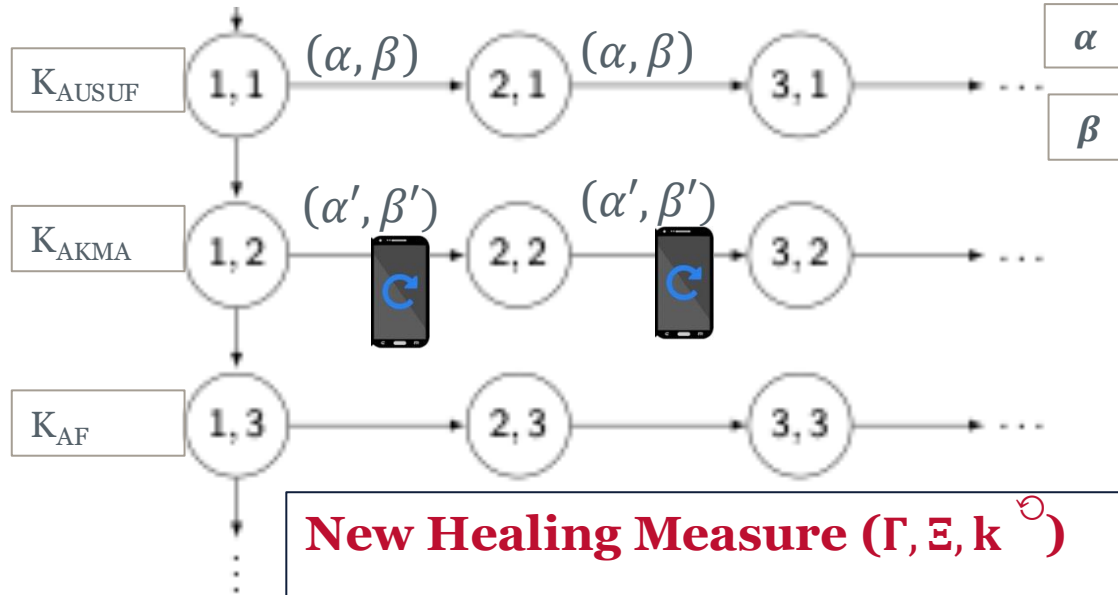
Key Hierarchy and Refreshing in 5G Registration [as per 3GPP 33.501]



Key Hierarchy and Refreshing in AKMA (application protocol)
[as per 3GPP 33.535]

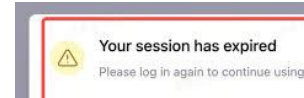


PCS Metric with **Application Controls**; Idea ...: Composable Extension of the PCS Metric



α Key-material control – [USENIX2023]

β **Application control** – NEW
*Keep a phone alive, restart it,
control a browser*



New Healing Measure (Γ, Ξ, k°)

(nb of $\Rightarrow$, nb of $\Downarrow$, controlling applications that stopping/starting keys' refresh)

(∞, ∞, K_{AF}) = PCS insecure forever, if K_{AF} appl. controlled

$(\infty, 1, K_{AF})$ = PCS healed “vertically” in 1 step, if K_{AF} appl. controlled

$(\infty, 2, K_{AKMA})$ = PCS healed “vertically” in 2 steps, if K_{AKMA} appl. controlled

- Computational model

$$\begin{array}{l}
 \text{Exp}_{\mathcal{A}}^{\text{PCS}}(\lambda, \mathcal{A}) \\
 (\tilde{s}, sk, \tilde{s}.pk, pparam) \leftarrow \text{GenSetup}(1^\lambda) \\
 (P = \{P_1, \dots, P_{np}\} = \mathcal{C} \cup \mathcal{S}) \leftarrow \mathcal{C}(\lambda, np) \\
 (P_i.pk_i, P_i.sk_i) \leftarrow \text{GenKeyGen}(1^\lambda) \quad \forall i \in \{1, \dots, np\} \\
 \mathcal{O}_{type} \leftarrow \{ \text{olReg}(\cdot), \text{oStart}(\cdot, \cdot, \cdot), \text{olReveal}[\mathcal{A}.reach](\cdot, \cdot), \text{oKE}(\cdot, \cdot, \cdot), \text{oH}[\text{ack}[\mathcal{A}.breadth]](\cdot, \cdot, \cdot, \cdot) \}; \\
 (\pi_p^*, e^*) \leftarrow \mathcal{A}^{\mathcal{O}_{type}}(1^\lambda) \\
 K \leftarrow \text{olTest}_{sk_p}(\pi_p^*, e^*) \\
 d \leftarrow \mathcal{A}^{\mathcal{O}_{type}}(\lambda, np, K) \\
 \boxed{\mathcal{A} \text{ wins iff. } d = e^* \text{ and } (\neg \text{olReg}(P) \vee \neg \text{olReg}(\pi_p^*.pid)) = \top} \\
 k \cup \perp \leftarrow \text{Confidentialize}(\cdot, \cdot)
 \end{array}$$

- Due to discrete measures (i.e., counting), PCS framework mechanised in ProVerif and in Tamarin
- In early theory for integers in ProVerif 2.04



Modelling PCS_AC in ProVerif:

making the most of integers' theories

right before launching ProVerif 2.04 ; then done in ProVerif 2.05; a story 😊 ...



Unbounded key derivation over a key hierarchy

K1

|

K2

|

K3, with K3 derived from K2, K2 from K1.

Unbounded horizontal key-evolutions at every level.



Integer-indexed key lists

A user-defined list theory over ProVerif's built-in `nat` type:

K1_list, K2_list, K3_list.

Kp_list[i] holds the i-th value of Kp.



Application refresh triggers

aEvolve oracle modelled as Booleans:
 α (e.g., for K3) and β (e.g., for K2), or variants..

α = True to evolve K3; β = True to evolve K2;
K1 refreshes every round.



letfun refresh functions

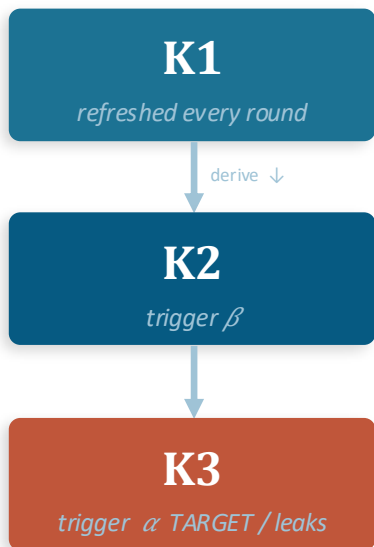
Per-level `letfun` uses α/β and the KDFs to either derive a fresh key or look up the old value, then appends it back to the list.

[FILE](#)

Leaking K_3 Keys & Verifying Healing

K_3 is the target key (“lowest-level” key, i.e., most attacker control).

It leaks at a fixed, user-defined point n : $K3_list[n]$.



nat-indexed queries (as per our computational def. of PCS)

- ✓ Executability: multiple key-evolution rounds occur (round $i+1 \Rightarrow$ round i).
- ✓ Leak trace exists: K_3 created & leaked at stage $i \Rightarrow$ attacker knows it.
- ✓ α -control: if attacker holds α , K_3 stays stale post-leak — known at later stages.
- ✓ β -control: with the non-fresh KDF, holding β keeps a refreshed K_3 unchanged.
- ✓ List provenance: K_3 known at position $i+1 \Rightarrow$ it is the very key leaked at position i .
- ✓

Result all queries · unbounded horizontal evolutions · < **8 min** on a commodity i7 laptop.

Why ProVerif was the best fit

We modelled this in Tamarin too.

Both tools modelled PCS_AC (very differently!!) and proved their queries.

For this counter-heavy, stateful problem, ProVerif gave a much more natural encoding.

ProVerif 2.04/2.05

- ✓ **User-defined lists + `letfun`** Natural model of any key-hierarchy & its evolution.
- ✓ **Native `nat` & `if-then-else`** Switching key-refresh options is trivial.
- ✓ **Readable, compact model** No duplicated rules; conditionals do the work.
- ✓ **ProVerif's resolution even with integers** Avoided the exponential blow-up of concrete indices.

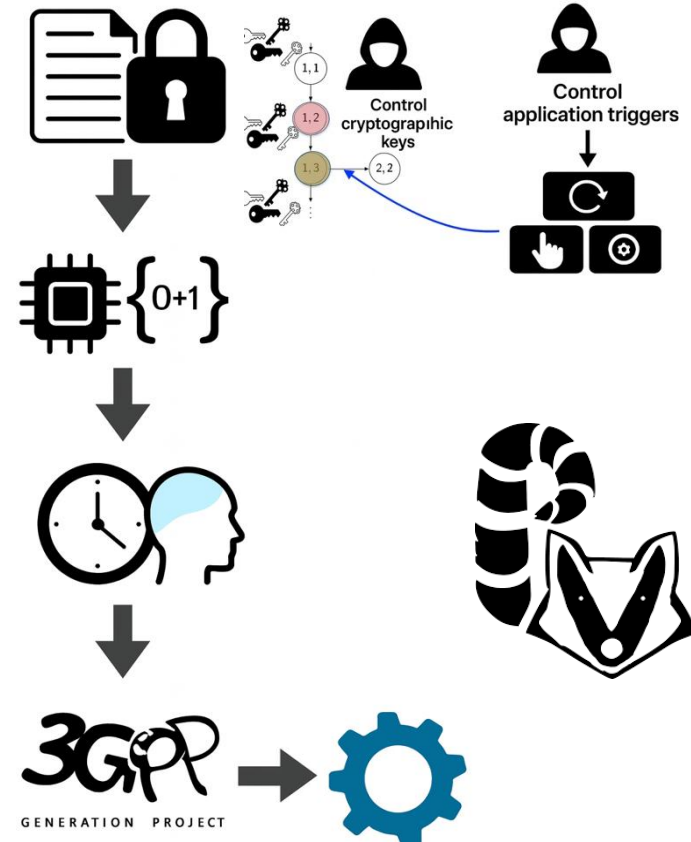
Tamarin 1.8.0

- ✗ **No lists or conditionals** Tracked evolutions via rules & state facts instead.
- ✗ **Branch-per-option rules** Duplicated code; harder-to-read model.
- ✗ **Needs a user-supplied oracle** One proof requires manual guidance.
- ⚡ **Multi-threaded — faster** Outperforms on raw runtime (~40 s).

Takeaway: Tamarin won on speed, but ProVerif's `letfun`, native `nat` and `if-then-else` gave a more natural encoding; our preferred choice for mechanising PCS_AC!

Conclusions

- ProVerif did *beautifully* at **approximating a PCS computational/cryptographic metric** that used counting discrete-math
- Used in **general for PCS**, but also for 5G AKMA & TLS with session tickets;
- Showed **5G AKMA PCS-insecure**, **5G AKMA+ PCS-secure** (for the same attacker) → **symbolical PCS-metric proofs**
- **Influencing 3GPP** revisions with the new AKMA+ protocol, formally verified; ProVerif "knows" a lot about that 😊
- **Long live ProVerif !**



Workshop: 25 years of ProVerif



June 3, 2026

THANK YOU!

Check out the full details in our ASIACCS2025 paper



The copyright of certain figures/icons/pictures used within is not attributed to the presenter.